

ForeSight: Small Calibrated Decision Models as Opponent Forecasters for Lookahead Game Play (Student Abstract)

Saiakhil Chilaka

Cornell University
sc3484@cornell.edu

Abstract

Choosing well against an opponent requires forecasting how the opponent will respond. Hand-built game engines forecast with expert rules and exact simulators, which do not exist for less deterministic settings such as negotiation, while LLM agents forecast with large generative models that are slow and poorly calibrated. We propose ForeSight, which fine-tunes a small non-generative “System One” decision model to output a calibrated probability for each opponent action, and uses these forecasts to weight every branch of a lookahead search. In Pokémon Showdown, a 421M-parameter forecaster fine-tuned for 35 minutes on a laptop, using only low-quality bot games, predicts the opponent’s next action with 72–80% accuracy (versus 43% zero-shot and 49% for a damage-based rule), and 75% on an opponent it never saw in training, at 36–84 ms per forecast. Without any LLM, ForeSight wins 56.5% of 200 games against the Abyssal benchmark bot, on par with the LLM agent PokéLLMon (56%), though below PokéChamp (64–70%). Because every component is still minimal, these results suggest substantial headroom for small calibrated forecasters to bring engine-style lookahead where engines cannot be hand-written.

Introduction

Many decisions, from game moves to negotiation offers, are only as good as the agent’s forecast of how the other side will react. Classical game engines make these forecasts with search over an exact simulator and an expert-written evaluation; the Pokémon engine Foul Play pairs tree search with a hand-tuned position score (Mariglia 2026a). This works where the rules are fully specified, but it does not transfer to settings such as negotiation, where there is no simulator and no obvious way to hand-write an opponent model.

LLM agents take the opposite route. PokéLLMon prompts an LLM to choose each move (Hu, Huang, and Liu 2024), and PokéChamp uses an LLM to sample actions, model the opponent and score positions inside a minimax search (Karten, Nguyen, and Jin 2025). LLMs bring broad knowledge, but each call takes seconds, their stated probabilities are poorly calibrated, and PokéChamp attributes about a third of its losses to running out of turn time.

To overcome these challenges, we propose ForeSight. Unlike prior approaches, ForeSight separates *forecasting* from

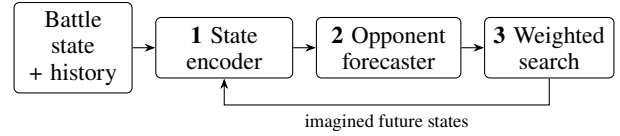


Figure 1: ForeSight. The forecaster assigns a probability to each opponent action; the search weights every imagined future by these probabilities and by chance, then picks the best expected move.

deciding: a small System One decision model (TypeSafe AI 2026; Convai Innovations 2026), which reads a state and a set of typed options and returns calibrated probabilities in a single forward pass without generating text, forecasts the opponent, and a probability-weighted search turns those forecasts into a decision.

ForeSight

ForeSight has three components (Figure 1).

1. State encoder. Each position is rendered as short pre-computed facts rather than raw text: remaining units and HP, which side moves first, estimated damage and knock-out chances for each attack, type effectiveness, and both sides’ last few actions. Candidate opponent actions are the revealed moves, one placeholder attack per type for unrevealed moves, and switches to seen Pokémon.

2. Opponent forecaster. We fine-tune Laya (Convai Innovations 2026), an open 421M-parameter System One model, on 10k opponent decisions sampled from 142k logged in local battles between simple rule-based and random bots, a deliberately low-quality data source. We update the top four encoder layers and the decision head (75M parameters) with Laya’s proper-scoring-rule objective and fit a temperature on validation data. Training takes 35 minutes on an Apple M5 laptop.

3. Probability-weighted search. ForeSight runs a depth-2 expectimax. For each of our actions it branches over the forecaster’s opponent distribution and over chance outcomes (accuracy, damage rolls near a knock-out, secondary effects, speed ties), pruning each node to 90% of its probability mass. Leaf positions are scored by remaining-HP balance, and the action with the highest expected value is played.

Forecaster	Evaluated on	Acc.	Log-loss
Damage-based rule	Gen 9 bots	48.5%	1.24
Laya, zero-shot	Gen 9 bots	43.4%	1.15
ForeSight (Gen 9)	Gen 9 bots	71.8%	0.70
ForeSight (Gen 8)	Gen 8 bots	79.6%	0.54
ForeSight (Gen 9)	Abyssal, unseen	73.0%	0.62
ForeSight (Gen 8)	Abyssal, unseen	75.4%	0.58

Table 1: Next-action forecasting on held-out turns ($n=3,754-4,000$ bot turns; 1,963 Abyssal turns).

Agent	Decision model	Win rate
PokéLLMon [†]	GPT-4o	56%
PokéChamp [†]	Llama-3.1-8B	64%
PokéChamp [†]	GPT-4o	70%
ForeSight (Gen 9)	Laya 421M	56.5%
ForeSight (Gen 8)	Laya 421M	56.5%

Table 2: Win rate against Abyssal. [†]Published by Karten, Nguyen, and Jin (2025). ForeSight: $n=200$ games each, 95% CI 50–63%.

Evaluation

Experimental Setup. We evaluate Gen 9- and Gen 8-trained forecasters on held-out battles, against a rule favoring the opponent’s strongest attack. For play, we follow PokéChamp’s protocol: Gen 8 random battles without Dynamax against the rule-based Abyssal bot, which never appears in our training data, on a local Showdown server with a 15 s turn limit. We compare with the win rates published by Karten, Nguyen, and Jin (2025).

Forecasting. Fine-tuning turns a weak zero-shot model into an accurate, well-calibrated forecaster (calibration error 0.02; Table 1) that transfers to Abyssal, never seen in training, predicting 73–75% of its actions.

Play. With no LLM, ForeSight performs on par with PokéLLMon while using a model a fraction of its size, though below PokéChamp (Table 2). The Gen 8 forecaster predicts better yet wins at the same rate; since forecasts on Abyssal are already accurate, the gap to PokéChamp likely lies more in our shallow search and simple leaf.

Speed and depth. A forecast costs 36–84 ms depending on batch size, and a turn takes 2.7–4.1 s on average (about 15 forecasts and 73 search nodes), so the 15 s budget would allow 10–25 $\times$ more search than we use. Depth pays off: with a fixed rule-based forecaster, win rate against a heuristic bot rises from 56% to 61% to 66% at depths 1–3 ($n=200$ each). The idea also extends beyond games: in the text household benchmark ALFWorld (Shridhar et al. 2021), letting a System One model choose among an LLM’s top three actions by predicted outcome raised success from 34.6% to 48.4% ($p<0.001$).

Conclusion and Future Work

We proposed ForeSight, which uses a small, fine-tuned, calibrated decision model as the opponent forecaster inside a

lookahead search. Even in this deliberately minimal form, it matches an LLM agent built on a much larger model. Zero-shot System One models (Laya, and Jev in our pilots) were weak forecasters, so domain fine-tuning is essential.

Headroom. Every component was kept simple, and each has a clear path to improvement backed by our results: (1) *Deeper search*: we use under a tenth of the turn budget, and win rate grew with every added level of depth. (2) *Better data*: the forecaster was trained only on weak bot games yet already predicts an unseen opponent 75% of the time; replays of high-rated human games (Grigsby et al. 2025) should sharpen it further. (3) *A learned leaf*: positions are scored by HP alone, so a setup move such as Swords Dance counts only if its extra damage lands within two turns; a learned win-probability leaf can value boosts, status and hazards. (4) *Full fine-tuning and exact rules*: we updated only 18% of the parameters, and an exact engine (Mariglia 2026b) with sampled hidden teams would remove known simulator errors. Because forecasts on Abyssal are already accurate, we expect (1), (3) and (4) in particular to narrow the gap to PokéChamp. Beyond games, ForeSight needs no hand-written opponent model, so we will extend it to negotiation (Lewis et al. 2017), where an LLM proposes candidate next states and the forecaster weighs them.

References

- Convai Innovations. 2026. Laya: A System 1 Decision Engine. <https://github.com/NandhaKishorM/laya>.
- Grigsby, J.; Xie, Y.; Sasek, J.; Zheng, S.; and Zhu, Y. 2025. Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers. ArXiv:2504.04395.
- Hu, S.; Huang, T.; and Liu, L. 2024. PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models. ArXiv:2402.01118.
- Karten, S.; Nguyen, A. L.; and Jin, C. 2025. PokéChamp: an Expert-level Minimax Language Agent. In *International Conference on Machine Learning*.
- Lewis, M.; Yarats, D.; Dauphin, Y.; Parikh, D.; and Batra, D. 2017. Deal or No Deal? End-to-End Learning of Negotiation Dialogues. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2443–2453.
- Mariglia, P. 2026a. Foul Play: A Pokémon Showdown Battle Bot. <https://github.com/pmariglia/foul-play>.
- Mariglia, P. 2026b. poke-engine: A Pokémon Battle Engine for Search. <https://github.com/pmariglia/poke-engine>.
- Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *International Conference on Learning Representations*.
- TypeSafe AI. 2026. Introducing System One Models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>.